

Lecture 3: Computer Architecture vs Computer Organization

Learning Objectives

By the end of this lecture, students should be able to:

- Define Computer Architecture.
 - Define Computer Organization.
 - Understand the difference between architecture and organization.
 - Understand what ISA means.
 - Identify programmer-visible features.
 - Understand how the same architecture can have different implementations.
 - Relate architecture and organization to CPU, memory, and assembly language.
-

1. Introduction



Teaching Script

Hello everyone, and welcome back to our **Computer Organization and Assembly Language** course.

In Lecture 1, we introduced Computer Organization, Computer Architecture, and Assembly Language.

In Lecture 2, we discussed the basic components of a computer system, including the CPU, ALU, Control Unit, registers, memory, input/output devices, and system buses.

Today, we're going to discuss one of the most important concepts in this entire course:

What is the difference between Computer Architecture and Computer Organization?

You will see these two terms again and again throughout this course.

Many students think they mean exactly the same thing.

They are closely related, but they are not the same.

So in today's lecture, we're going to understand exactly what each term means, how they are different, and how they work together.

Let's begin.

2. What is Computer Architecture?

Let's start with **Computer Architecture**.

Definition

Computer Architecture describes the functional behavior and programmer-visible characteristics of a computer system.

In simple words:

Computer Architecture tells us WHAT the computer does.

It defines things that a programmer needs to know in order to write software for the computer.

These include:

- Instruction set
 - Registers available to programmers
 - Data types
 - Instruction formats
 - Addressing modes
 - Memory addressing
 - Input/output mechanisms
-

3. What is ISA?

One of the most important concepts related to architecture is **ISA**.

ISA stands for:

Instruction Set Architecture

The ISA defines the instructions that a processor understands.

For example, an architecture might support instructions such as:

ADD
SUB
LOAD
STORE
JUMP
BRANCH

The ISA also defines things such as:

- Registers
- Instruction formats
- Data types
- Addressing modes
- Memory model

Simple Example

Imagine that a CPU supports:

ADD \$t0, \$t1, \$t2

The fact that this instruction exists and what it means is part of the **architecture/ISA**.

How the CPU internally performs that addition is an **organization** issue.

4. What is Computer Organization?

Now let's define organization.

Computer Organization describes how the hardware components are arranged and implemented to realize the computer architecture.

In simple words:

Computer Organization tells us HOW the computer does it.

Organization deals with things such as:

- ALU implementation

- Control unit implementation
 - Internal CPU connections
 - Buses
 - Cache organization
 - Memory technology
 - Control signals
 - Hardware implementation
-

5. The Most Important Difference

Give students this simple rule:

Architecture = WHAT

Organization = HOW

For example:

Suppose the architecture says:

The processor supports an ADD instruction.

That's **architecture**.

But:

How does the ALU perform the addition?

How are the registers connected to the ALU?

How are control signals generated?

These are **organization** questions.

6. Easy Real-Life Example — Car

This analogy is very useful.

Imagine two cars.

Both cars have:

- 4 wheels
- Steering
- Brakes
- Seats
- Engine
- Similar maximum speed

These are characteristics that describe **what the car provides**.

Now look inside.

Car A might have:

- One type of engine
- Different transmission
- Different wiring
- Different internal components

Car B might use:

- Another engine
- Different transmission
- Different internal components

Both can provide similar functionality, but their internal implementations can be different.

Similarly, two computer systems can implement the same architecture using different hardware organizations.

7. Architecture vs Organization Table

This is an important exam table.

Computer Architecture	Computer Organization
What the system does	How the system does it
Programmer-visible	Mostly implementation-level
Defines ISA	Implements ISA

Instruction formats	Control signals
Data types	Hardware components
Addressing modes	Internal connections
Programmer-visible registers	ALU implementation
Memory addressing model	Cache organization

8. Another Simple Example

Suppose a computer supports:

ADD
SUB
MUL
DIV

The fact that these instructions are supported is part of the **architecture**.

Now ask:

Architecture question:

Does the processor support multiplication?

That's architecture.

Organization question:

How is multiplication implemented inside the processor?

That's organization.

It could be implemented using dedicated hardware, iterative circuitry, or other internal techniques.

The programmer doesn't necessarily need to know the internal implementation to use the instruction.

9. Programmer's Perspective

This is an important concept.

A programmer might write:

```
ADD $t0, $t1, $t2
```

The programmer needs to know:

- What **ADD** does
- Which registers are involved
- What format the instruction uses
- What result it produces

These are architecture-level concerns.

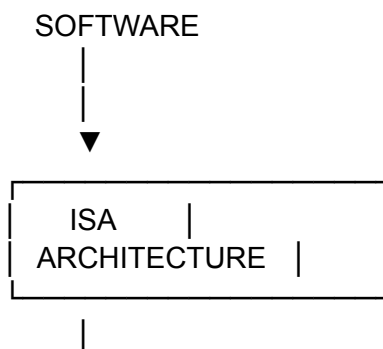
The programmer usually doesn't need to know:

- How individual transistors implement addition
- How the ALU is physically wired
- Exactly how control signals travel inside the processor

Those are organization/implementation concerns.

10. Architecture as an Interface

You can think of architecture as an **interface between software and hardware**.



▼
HARDWARE
ORGANIZATION

The ISA provides the rules that software uses to communicate with the processor.

For example:

Software says:
"Execute ADD."

Architecture defines:
"What ADD means."

Organization determines:
"How ADD is physically executed."

This is an excellent way for students to remember the concept.

11. What Does Architecture Define?

Let's look at the major elements.

11.1 Instruction Set

Architecture defines what instructions are available.

For example:

ADD
SUB
LOAD
STORE
JUMP
BRANCH

11.2 Registers

Architecture defines which programmer-visible registers exist.

For example, MIPS has registers such as:

\$t0
\$t1
\$s0
\$s1
\$a0
\$v0

The exact register set depends on the architecture.

11.3 Data Types

Architecture may define the data types supported by instructions.

For example:

- Integers
 - Bytes
 - Characters
 - Floating-point values
-

11.4 Addressing Modes

Architecture can define how instructions specify operands.

For example:

Immediate
Register
Base/Displacement

We'll study addressing modes in much greater detail later.

11.5 Instruction Formats

Architecture defines how instructions are structured.

For example, MIPS has:

R-type

I-type

J-type

We'll study these later in the course.

12. What Does Organization Define?

Now look at organization.

Organization deals with the hardware implementation.

Examples include:

ALU

How arithmetic and logical operations are implemented.

Control Unit

How control signals are generated.

Registers

How registers are physically implemented and connected.

Buses

How internal components communicate.

Cache

How cache memory is organized.

Memory System

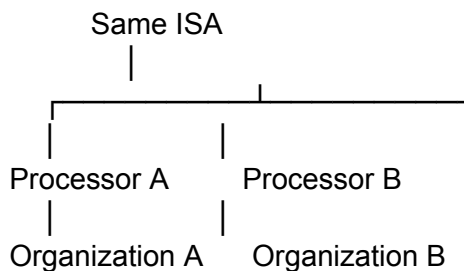
How memory components interact with the processor.

13. Same Architecture, Different Organization

This is one of the most important concepts.

Two processors can support the **same ISA** but have different internal designs.

For example:



Both processors can understand the same instructions.

But internally they may have different:

- Cache sizes
- Pipeline designs
- Control units
- ALUs
- Clock speeds
- Number of execution units
- Internal connections

Therefore:

Same architecture does not necessarily mean same organization.

14. Example — MIPS

Since we'll use MIPS throughout the assembly section, let's connect today's concept to MIPS.

MIPS defines things such as:

- Registers
- Instruction formats
- Instruction set
- Addressing
- Data types

These are architecture-level concepts.

But the actual processor implementation contains:

- ALU
- Control Unit
- Registers
- Multiplexers
- Buses
- Memory interface
- Pipeline
- Cache

These are organization-level concepts.

15. Architecture vs Organization vs Implementation

Students sometimes encounter a third term:

Implementation

You can think about the three levels like this:

Architecture



WHAT the computer should do

Organization



HOW the components are arranged

Implementation



HOW the physical hardware realizes it

In some textbooks, **organization and implementation** are discussed together or with slightly different boundaries.

So don't get confused if different books use these terms somewhat differently.

The central idea remains:

Architecture defines the programmer-visible behavior, while organization focuses on the internal hardware arrangement that implements it.

16. Example — Addition

Let's take a complete example.

Suppose the programmer writes:

ADD \$t0, \$t1, \$t2

Architecture

Defines:

ADD instruction exists.

It adds two register values.

The result goes into the destination register.

Organization

Deals with:

Registers
↓
ALU
↓
Control signals
↓
Result

The internal hardware performs the actual operation.

17. Why Is This Difference Important?

Understanding this distinction helps us understand:

Software Compatibility

Programs are designed for specific instruction sets.

Processor Design

Hardware designers need to implement the architecture.

Performance

Different organizations can provide different performance even when the architecture is similar.

Cost

Different hardware implementations can have different costs.

Power Consumption

Different organizations can consume different amounts of energy.

18. Architecture Determines Compatibility

This is an important practical concept.

Suppose software is compiled for one instruction set.

A processor that doesn't support that instruction set cannot necessarily execute the program directly.

For example, software compiled for one ISA may require translation or recompilation to run on another ISA.

So:

ISA



Instruction compatibility



Software compatibility

This is one reason architecture matters so much.

19. Performance and Organization

Suppose two processors support the same instruction set.

Processor A → Same ISA

Processor B → Same ISA

But:

Processor A → smaller cache

Processor B → larger cache

Processor A → simpler pipeline

Processor B → more advanced pipeline

Their performance can be different.

So even if the architecture is the same, organization can affect:

- Speed
- Power
- Cost

- Efficiency
-

20. Simple Analogy — Restaurant

Here's another analogy students usually understand quickly.

Imagine a restaurant menu.

The **menu** tells customers what they can order.

That is similar to **architecture**.

For example:

Burger
Pizza
Pasta

But the menu doesn't tell you exactly how the kitchen prepares the food.

The kitchen's:

- Equipment
- Layout
- Staff
- Cooking process

are similar to **organization**.

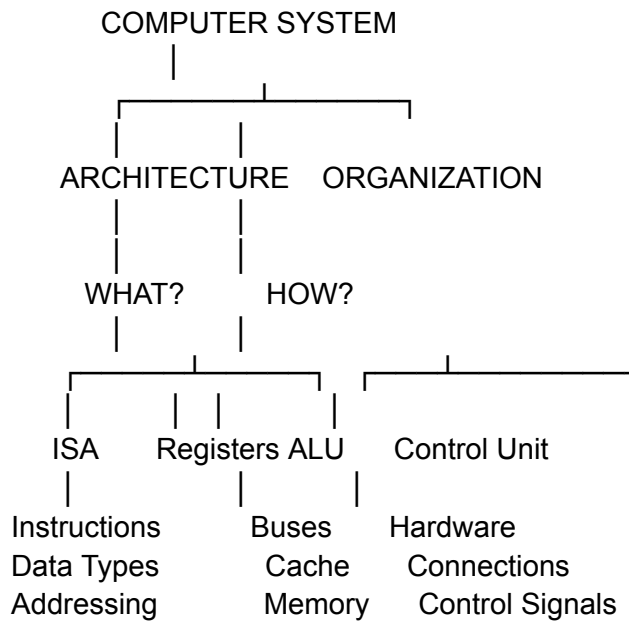
So:

Architecture = What is offered

Organization = How it is produced

21. Complete Conceptual Diagram

Use this diagram during your lecture:



Lecture Closing Script

So, let's summarize today's lecture.

Today we learned the difference between **Computer Architecture** and **Computer Organization**.

Computer Architecture describes the **programmer-visible characteristics** of a computer system. It includes things like the instruction set, registers, data types, addressing modes, and instruction formats.

Computer Organization describes **how the hardware is arranged and implemented** to support that architecture. It includes the ALU, Control Unit, buses, cache, memory organization, and control signals.

The easiest way to remember the difference is:

Architecture tells us WHAT the computer does.

Organization tells us HOW the computer does it.

We also introduced **Instruction Set Architecture, or ISA**, which acts as an interface between software and hardware.

In the next lecture, we are going to study one of the most important computer architecture models: **Von Neumann Architecture**.

We'll understand how programs and data are stored in memory, how the CPU communicates with memory, and why the Von Neumann model became the foundation of many modern computer systems.

Thank you for watching, and I'll see you in the next lecture.
